

ChatGPT - OS

Write a program where a parent process creates two child processes. The first child computes the factorial of a given number and the second child generates the Fibonacci series up to N terms. in c

The program below satisfies the assignment:

Parent process creates two child processes.

- **Child 1:** Computes the factorial of a given number.
- **Child 2:** Generates the Fibonacci series up to **N** terms.
- The **parent waits** for both child processes to complete using **wait()**.

This matches **Question 2 – Process Management** from your assessment.

SYSTEM PROGRAMMING ASSESSMENT

C Program

```
C
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int n;
    pid_t pid1, pid2;

    printf("Enter a number: ");
    scanf("%d", &n);

    // Create first child process
    pid1 = fork();

    if (pid1 < 0)
    {
        printf("First fork failed!\n");
        exit(1);
    }

    if (pid1 == 0)
    {
        // First Child Process - Factorial

        long long fact = 1;

        for (int i = 1; i <= n; i++)
        {
            fact = fact * i;
        }

        printf("\n----- First Child Process ----- \n");
        printf("Child-1 PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Factorial of %d = %lld\n", n, fact);
    }
}
```

```

        exit(0);
    }

    // Create second child process
    pid2 = fork();

    if (pid2 < 0)
    {
        printf("Second fork failed!\n");
        exit(1);
    }

    if (pid2 == 0)
    {
        // Second Child Process - Fibonacci

        int a = 0, b = 1, c;

        printf("\n----- Second Child Process ----- \n");
        printf("Child-2 PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("Fibonacci Series (%d terms): ", n);

        for (int i = 1; i <= n; i++)
        {
            printf("%d ", a);
            c = a + b;
            a = b;
            b = c;
        }

        printf("\n");

        exit(0);
    }

    // Parent waits for both children
    wait(NULL);
    wait(NULL);

    printf("\n----- Parent Process ----- \n");
    printf("Parent PID : %d\n", getpid());
    printf("Both child processes have completed.\n");

    return 0;
}

```

Compilation

```

Bash
gcc program.c -o program

```

Execution

```

Bash
./program

```

Sample Output

```

Enter a number: 6

----- First Child Process -----
Child-1 PID : 4521
Parent PID : 4520

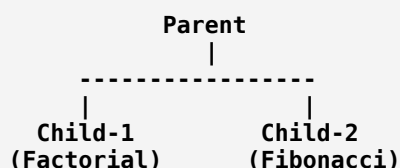
```

```
Factorial of 6 = 720

----- Second Child Process -----
Child-2 PID : 4522
Parent PID  : 4520
Fibonacci Series (6 terms): 0 1 1 2 3 5

----- Parent Process -----
Parent PID : 4520
Both child processes have completed.
```

Process Tree



System Calls Used

System Call	Purpose
<code>fork()</code>	Creates a child process. Two calls create two child processes.
<code>getpid()</code>	Returns the current process ID.
<code>getppid()</code>	Returns the parent process ID.
<code>wait()</code>	Parent waits until a child process terminates.
<code>exit()</code>	Terminates the child process after completing its task.

This implementation uses **two separate `fork()` calls**, resulting in one parent process and two sibling child processes, with the parent synchronizing their completion using `wait()`.

SYSTEM PROGRAMMING ASSESSMENT

Your understanding is already very close. Let me refine a few points and explain the parts that are still confusing, especially `fflush(stdout)` and `perror()`, because these are concepts that many students struggle with.

1. If I don't use `exit()`, the child creates a grandchild. Can I use `return 0` instead?

Yes. Inside `main()`, you can write either:

```
C
return 0;
```

or

```
C
exit(0);
```

Both terminate the **current process**.

Example:

```
C
if(pid == 0)
{
    printf("Child Process\n");
    return 0;    // Child process terminates here
}
```

This is equivalent to

```
C
if(pid == 0)
{
    printf("Child Process\n");
    exit(0);    // Child process terminates here
}
```

So your understanding is correct.

Why do many Linux programmers use `exit()`?

Because sometimes the child code is **not inside** `main()`.

Example

```
C
void factorial()
{
    printf("Calculating...\n");
    exit(0);    // Entire process terminates
}
```

If you write

```
C
return;
```

only the function returns.

```
main()
{
    factorial();
    printf("Still executing...");
}
```

Output

```
Calculating...
Still executing...
```

because only the function ended.

But

```
exit(0);
```

terminates the whole process immediately.

2. EXIT_SUCCESS and EXIT_FAILURE

You said

if OS changes it, then only problem, until then no issue.

Exactly.

On Linux,

```
EXIT_SUCCESS = 0
EXIT_FAILURE = 1
```

almost always.

So these are identical:

```
C
return 0;
```

```
C
return EXIT_SUCCESS;
```

and

```
C
return 1;
```

```
C
return EXIT_FAILURE;
```

produce the same result **on Linux**.

The reason we prefer **EXIT_SUCCESS** is **readability and portability**.

When another programmer reads

```
C
return EXIT_FAILURE;
```

they instantly know:

The program failed.

instead of wondering

Why is it returning 1?

3. Now let's understand `fflush(stdout)` deeply

This is probably the hardest concept.

We first need to understand **buffering**.

Suppose you write

```
C
printf("Hello");
```

Question:

Does `printf()` immediately print on the screen?

Most beginners think

Yes.

Actually,

No.

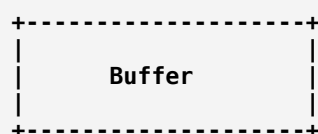
It usually writes into a **buffer** first.

What is a buffer?

A buffer is just a temporary memory area.

Suppose

Memory



Now

```
C
printf("Hello");
```

does

Buffer

```
+-----+
| Hello |
+-----+
```

NOT

```
Screen
Hello
```

Why?

Printing to the monitor is slow.

Memory is very fast.

Therefore

```
printf()
↓
Memory Buffer
↓
Later
↓
Screen
```

This improves performance.

When is the buffer emptied?

Usually

Case 1

Buffer becomes full.

```
Buffer
AAAAAAAAAAAAAAAAAAAAAA
↓
Automatically printed.
```

Case 2

You print

```
C
printf("\n");
```

Because stdout connected to terminal is line-buffered, the newline often flushes the buffer.

Case 3

You manually flush.

```
C
fflush(stdout);
```

Example

```
C
printf("Hello");
fflush(stdout);
```

Immediately

```
Buffer
Hello
↓
Screen
Hello
```

Now the buffer becomes empty.

```
Buffer
(empty)
```

Now combine this with fork()

Suppose

```
C
printf("Hello");
fork();
```

At this moment

```
Buffer
Hello
```

has **not** been printed yet.

Now

```
fork()
```

copies the entire process memory.

Parent

```
Buffer
Hello
```

Child

```
Buffer
Hello
```

Now there are **two copies**.

Later,

both processes terminate.

When they terminate,

the C library automatically flushes stdout.

Parent prints

```
Hello
```

Child prints

```
Hello
```

Final output

```
HelloHello
```

even though you called

```
C
printf()
```

only once.

Diagram

Before fork

```
Parent
Buffer
Hello
```

After fork

Parent Buffer	Child Buffer
Hello	Hello

Later

Parent	
prints Hello	
Child	
prints Hello	

Output

HelloHello

Solution

Before

C
<code>fork();</code>

write

C
<code>fflush(stdout);</code>

Now

Parent
Buffer
(empty)

Then

<code>fork()</code>

copies

Parent Buffer
(empty)
↓
Child Buffer
(empty)

Nothing gets duplicated.

Another way

If you write

```
C
printf("Hello\n");
```

the newline usually flushes the buffer automatically when stdout is connected to a terminal.

Therefore

```
Buffer
(empty)
```

before fork.

No duplication.

4. What exactly is perror()?

This is another Linux-specific feature.

Suppose

```
C
FILE *fp = fopen("abc.txt", "r");
```

But

```
abc.txt
```

doesn't exist.

Internally,

the operating system stores the reason in a global variable called **errno**.

Conceptually,

```
errno
↓
ENOENT
```

which means

```
No such file or directory.
```

Now suppose you write

```
C
```

```
printf("File open failed");
```

Output

```
File open failed
```

Question:

Why did it fail?

You don't know.

Maybe

- file doesn't exist
- permission denied
- disk error
- path invalid

No information.

Now use

```
C  
perror("File open failed");
```

Output

```
File open failed: No such file or directory
```

Notice

```
File open failed
```

came from **you**.

```
No such file or directory
```

came from the **operating system**.

How does perror() know?

Internally

```
Operating System
```

```
↓
```

```
errno
```

```
↓
```

ENOENT

`perror()` reads `errno` and converts it into a human-readable message.

Conceptually, it's like this:

```
C
printf("%s: %s\n", message, strerror(errno));
```

So

```
C
perror("fork failed");
```

might produce

```
fork failed: Resource temporarily unavailable
```

Here:

- "fork failed" is your message.
- "Resource temporarily unavailable" is the OS explanation.

Why not just use `printf()`?

You certainly can:

```
C
printf("fork failed\n");
```

Output

```
fork failed
```

But you still don't know the **reason**.

With

```
C
perror("fork failed");
```

Output

```
fork failed: Resource temporarily unavailable
```

That extra information is invaluable when debugging system calls like `fork()`, `open()`, `read()`, `write()`, `wait()`, `exec()`, etc.

Final Summary

Question	Answer
Can <code>return 0</code> replace <code>exit(0)</code> inside <code>main()</code> ?	Yes. Both terminate the current process when used in <code>main()</code> .
Why use <code>exit()</code> then?	It can terminate the entire process from any function , not just <code>main()</code> .
Can I use <code>1</code> instead of <code>EXIT_FAILURE</code> ?	Yes on Linux, but <code>EXIT_FAILURE</code> is clearer and portable across C implementations.
What is <code>fflush(stdout)</code> ?	It forces any buffered output waiting in <code>stdout</code> to be written immediately, preventing duplicated output after <code>fork()</code> .
Why use <code>perror()</code> instead of <code>printf()</code> ?	<code>printf()</code> only prints your message. <code>perror()</code> prints your message plus the operating system's actual error description stored in <code>errno</code> , making debugging much easier.

am still not undersatdn the fflush(stdout);

This is completely normal. `fflush(stdout)` **is not difficult, but most books explain it in one sentence, which makes it confusing.** Let's ignore `fork()` for a moment and first understand **how `printf()` actually works internally.**

Step 1: What do you think happens here?

```
C
printf("Hello");
```

Most beginners think:

```
printf("Hello")
↓
Screen
Hello
```

This is NOT what usually happens.

Step 2: What actually happens?

Before anything appears on the screen, C uses a **buffer**.

A **buffer** is simply a temporary memory area.

Imagine RAM contains this:

```
+-----+
|          |
|  BUFFER  |
|          |
+-----+
```

```
|
+-----+
```

Now execute

```
C
printf("Hello");
```

The data goes here:

```
+-----+
|          |
|  BUFFER  |
|          |
| Hello    |
|          |
+-----+
```

Notice:

Nothing has appeared on the screen yet.

The text is only stored in memory.

Step 3: Why use a buffer?

Suppose you write

```
C
for(int i=0;i<100000;i++)
{
    printf("A");
}
```

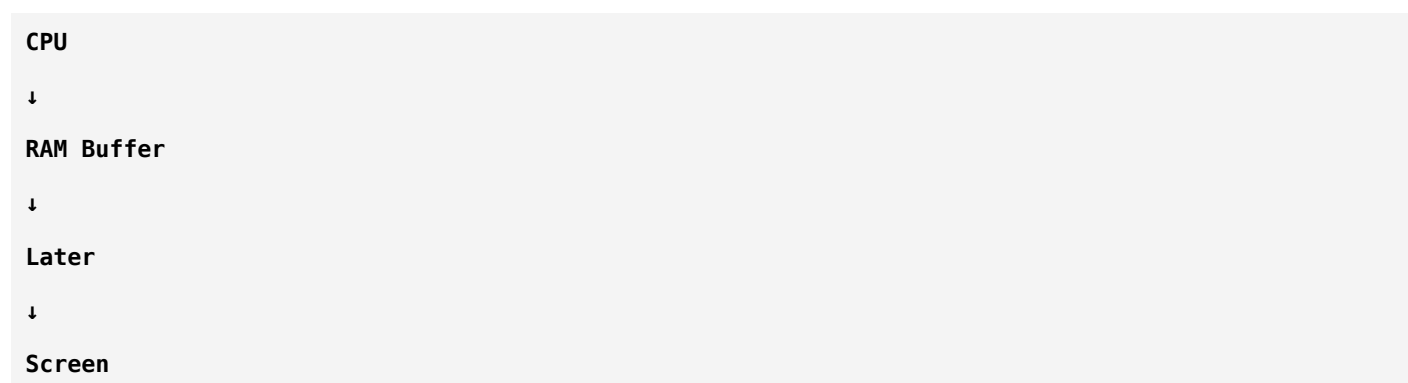
Without a buffer:

```
CPU
↓
Screen
↓
CPU
↓
Screen
↓
CPU
↓
Screen
↓
100000 times
```

Printing to the monitor is **very slow**.

Instead, C stores everything in RAM first.

RAM is extremely fast.



This is much faster.

Step 4: When does it actually print?

Suppose

```
C
printf("Hello");
```

Buffer

Hello

Screen

(empty)

Nothing happens until one of these occurs.

Case 1

Program ends.

```
return 0;
```

Before terminating,
the C library empties the buffer.

Now

Screen

Hello

Case 2

You print a newline

```
C
printf("Hello\n");
```

For terminals, the newline usually forces the buffer to be written.

Case 3

You manually say

```
C
fflush(stdout);
```

This means

"Whatever is waiting inside stdout, print it **RIGHT NOW**."

Example

```
C
#include<stdio.h>

int main()
{
    printf("Hello");
    fflush(stdout);
    while(1);
    return 0;
}
```

What happens?

```
printf()
↓
Buffer
Hello
```

Then

```
fflush(stdout)
```

Immediately

```
Screen
Hello
```

The program never ends because of

```
C
while(1);
```

but "Hello" is already visible because you manually flushed the buffer.

Without fflush()

```
C
#include<stdio.h>

int main()
{
    printf("Hello");
    while(1);
    return 0;
}
```

What happens?

```
Buffer
Hello
```

Program never ends.

So buffer never empties.

Result:

```
Screen
Nothing
```

You don't see "Hello" because it is still sitting in the buffer.

Now let's bring fork() into the picture

Consider this program:

```
C
printf("Hello");
fork();
```

Before fork()

Memory looks like this

```
Parent Process
-----
Code
```

```
Stack
Heap

stdout Buffer

Hello
-----
```

Notice:

The buffer still contains

```
Hello
```

It has **not** been printed yet.

Then fork()

Remember what `fork()` does.

It copies the entire process.

```
Parent
↓
fork()
↓
Parent      Child
Buffer      Buffer
Hello      Hello
```

Now there are **two independent buffers**.

One belongs to the parent.

One belongs to the child.

Program finishes

When a process finishes,

C automatically flushes stdout.

Parent flushes

```
Hello
```

Child flushes

```
Hello
```

Screen

```
HelloHello
```

You printed only once.

But got

```
HelloHello
```

because **both processes had their own copy of the same buffer.**

Visual Diagram

Before fork

```
Parent
stdout Buffer
+-----+
| Hello |
+-----+
```

After fork

```

      fork()
Parent      Child
+-----+   +-----+
| Hello |   | Hello |
+-----+   +-----+
```

Both terminate.

Both flush.

```
Parent prints
↓
Hello
Child prints
↓
Hello
Final Output
HelloHello
```

Now add fflush()

Program

```
C
printf("Hello");
fflush(stdout);
fork();
```

Step 1

```
Buffer
Hello
```

Step 2

```
fflush(stdout)
```

Immediately

```
Screen
Hello
```

Buffer becomes empty.

```
Buffer
(empty)
```

Now

```
fork()
```

copies

```
Parent Buffer
(empty)
↓
Child Buffer
(empty)
```

Both terminate.

Nothing more to print.

Output

```
Hello
```

Only once.

Think of the buffer like a notebook

Imagine the buffer is a notebook.

You write

```
Hello
```

inside the notebook.

```
Notebook
```

```
Hello
```

You haven't shown it to anyone yet.

Now someone photocopies the notebook.

```
Original Notebook
```

```
Hello
```

```
↓
```

```
Photocopy
```

```
Hello
```

Now there are **two notebooks**.

Both people later read their notebook aloud.

Person 1

```
Hello
```

Person 2

```
Hello
```

Everyone hears

```
HelloHello
```

If you **empty the notebook before photocopying**, then both notebooks are blank.

That's exactly what

```
C  
fflush(stdout);
```

does.

It **empties the stdout notebook by sending its contents to the screen before** `fork()` **duplicates the process.**

I can also show you this by writing two tiny programs (10 lines each)—one without `fflush(stdout)` that prints `HelloHello`, and one with `fflush(stdout)` that prints only `Hello`. Seeing the actual output usually makes the concept click immediately.

wihout use of it what will happen , do any malfunction happen while printing on screen

Excellent question. The answer is:

No, your program will usually not malfunction. `fflush(stdout)` is **not mandatory**. It is only needed in specific situations, especially around `fork()`.

Let's see exactly what happens.

Case 1: No `fork()`

Program:

```
C
#include <stdio.h>

int main()
{
    printf("Hello");
    return 0;
}
```

Output:

```
Hello
```

No problem.

Why?

When `main()` returns,

```
C
return 0;
```

the C runtime internally calls `exit(0)`.

Before terminating, `exit()` automatically flushes all output buffers.

So

```
printf("Hello")
↓
Buffer
Hello
```

```
↓
Program ends
↓
exit()
↓
Buffer flushed
↓
Screen
Hello
```

Everything works perfectly.

Case 2: With `fork()`, but using a newline

```
C
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Hello\n");
    fork();
    return 0;
}
```

Output is usually

```
Hello
```

only once.

Why?

Because

```
C
printf("Hello\n");
```

contains a newline.

When `stdout` is connected to a terminal, the newline usually causes the buffer to be flushed immediately.

So before `fork()`:

```
Buffer
(empty)
```

Nothing gets copied.

Case 3: With fork(), no newline, no fflush()

```
C
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Hello");
    fork();
    return 0;
}
```

Possible output:

```
HelloHello
```

Notice:

The program is **not wrong**.

It doesn't crash.

It doesn't hang.

It doesn't corrupt memory.

The only issue is:

The same buffered output is printed twice.

Why?

Before fork():

```
Parent
stdout buffer
Hello
```

fork() copies the entire process.

Now

```
Parent buffer
Hello
```

and

```
Child buffer
Hello
```

Both processes terminate.

When they terminate,

both automatically flush their own buffer.

Therefore

```
HelloHello
```

Is this a malfunction?

No.

It is called **duplicate buffered output**.

Nothing is broken.

The operating system is behaving exactly as designed.

Case 4: With `fflush(stdout)`

```
C
printf("Hello");
fflush(stdout);
fork();
```

Now

Before `fork()`:

```
Buffer
Hello
```

`fflush()`

↓

```
Screen
Hello
```

Buffer becomes

```
(empty)
```

Then

```
fork()
```

copies

Parent Buffer

(empty)

Child Buffer

(empty)

Nothing extra gets printed.

Output

Hello

When should you actually use `fflush(stdout)`?

Only when **all** of these are true:

- You printed something **without a newline**.
- That output is still in `stdout`'s buffer.
- You are about to call `fork()`.
- You don't want the child to inherit that buffered output.

Do most programs use it?

No.

Most programs simply write:

```
C
printf("Hello\n");
```

Since the newline usually flushes terminal output, there's no issue.

That's why many programmers rarely call `fflush(stdout)` before `fork()`.

In your assignment

Your program starts like this:

```
C
printf("Enter a number: ");
scanf("%d", &n);
```

Notice there is **no** `\n` after "Enter a number: ".

After `scanf()` reads the input, the prompt has normally already been displayed because `scanf()` on a terminal causes `stdout` to be flushed automatically before waiting for input. In this specific program, adding

```
C
fflush(stdout);
```

before `fork()` is **good defensive programming**, but the program will usually work correctly even without it.

Summary

Without <code>fflush(stdout)</code>	Result
No <code>fork()</code>	No problem
<code>fork()</code> after buffered output	Buffered text may be printed more than once
Program crash?	✗ No
Memory corruption?	✗ No
Wrong calculations?	✗ No
Duplicate output possible?	✓ Yes

So `fflush(stdout)` is **not used to fix a malfunction**. It is used to ensure that **any pending output is written exactly once before the process is duplicated by `fork()`**, preventing confusing duplicate output.